

CoralSwift — Complete Platform Workflow

CoralSwift Technologies is a single platform with five surfaces:

| Surface | Who uses it | What it does |

|---|---|---|

| Public Website | Visitors | Marketing pages, blogs, contact form, careers, SEO |

| Admin Panel | Staff roles | Centralized management of the whole system |

| Client Portal | Clients | Own projects, proposals, invoices, documents, tickets |

| Employee Portal | Employees | Attendance, leave, projects, timesheets, payslips |

| Partner Portal | Partners | Partner-specific projects and support tickets |

Backend: FastAPI, all endpoints under `/api/v1`.

This document synthesizes the requirements workflow doc (`docs/requirements/Complete\_Website\_Portal\_Workflow.md`) with the implemented system, verified against the code on 2026-09-30.

1. Roles

Defined in `app/models/enums.py` (`UserRole`):

`super\_admin`, `admin`, `hr`, `sales`, `marketing`, `project\_manager`, `developer`, `qa`, `support`, `finance`, `client`, `employee`, `guest`, `partner`

Role-based access control: every user only sees the modules and data their role allows (e.g. a sales user only converts their **own** leads; a client only ever sees their own records).

2. End-to-End Customer Lifecycle

flowchart TD

```
A["Visitor browses the public website"] --> B["Submits the contact form<br/>(rate-limited 5/minute)"]
B --> C["ContactSubmission stored – status: new"]
C --> D["Staff picks it up – status: in_progress<br/>marketing is notified"]
D --> E["Convert to Lead (POST /leads)<br/>links back to the submission"]
E --> F["Lead evaluation – sales / admin<br/>meeting · requirements · budget · timeline"]
F -->|"Reject / close"| G["Lead: disqualified"]
F -->|"Successful"| H["Convert Lead to Client<br/>sales: own leads only · admin/PM: any"]
H --> I["Client portal account provisioned<br/>+ credential email sent (Brevo)"]
I --> J["Client logs into the Client Portal"]
J --> K["Proposal drafted – lead: proposal_created"]
K --> L["PM review"]
L -->|"pm_rejected"| K
L -->|"pm_approved"| M["Proposal sent → viewed<br/>lead: proposal_sent"]
M --> N["Client decision"]
N -->|"Reject"| P["Revision (new version supersedes)<br/>or lead disqualified"]
N --> K
N -->|"Accept"| O["Lead: proposal_approved<br/>Project auto-created – status: planning"]
O --> Q["Contract drafted from accepted proposal<br/>pending → signed by both parties"]
Q --> R["PM assigns employees<br/>execution starts – status: in_progress"]
R --> S["Tasks · milestones · daily updates<br/>documents · meetings"]
S --> T["Project completed<br/>final deliverables shared with client"]
T --> U["Client approves the completed work"]
U --> V["Payment recorded (idempotent per transaction ref)"]
V --> W["Invoice generated<br/>available in the Client Portal"]
```

Stage 1 — Public Website → Contact Submission

1. Visitor browses the public site and submits the **Contact Form** (name, email, phone, company, industry, required service, requirements, budget) — rate-limited to 5/minute.
2. Submission is stored as a `ContactSubmission` (``status: new``) and a notification email goes to the team.
3. Contact statuses: ``new`` → ``in_progress`` → ``resolved`` (or ``spam``).
4. **Lead creation is a manual staff action**, by design: sales/marketing marks the submission ``in_progress`` (marketing is notified), then uses **Convert to Lead** (``POST /leads``, linking ``contact_submission_id``) → a CRM ``Lead`` is created.

Stage 2 — Lead Evaluation (sales / admin)

```
stateDiagram-v2
    [*] --> new
    new --> contacted
    contacted --> requirement_gathering
    requirement_gathering --> proposal_created : draft proposal created
    proposal_created --> proposal_sent : proposal emailed
    proposal_sent --> proposal_approved : client accepts
    proposal_approved --> converted : convert lead to client
    new --> disqualified : rejected
    contacted --> disqualified : rejected
    requirement_gathering --> disqualified : rejected
    proposal_sent --> disqualified : client rejects (if not yet converted)
    converted --> [*]
    disqualified --> [*]
```

- The evaluation record on the lead stores: ``evaluation_date``, ``meeting_notes``, ``requirements_confirmed``, ``delivery_timeline``, ``evaluation_result``, ``rejection_reason``.
- Two outcomes: **Convert Lead to Client** (Stage 3) or **Reject/Close** → ``disqualified`` (a disqualified lead can never be converted — the convert endpoint rejects it).

Stage 3 — Lead → Client Conversion

- Performed by an authorized user: sales (own leads only) or admin/project_manager (any lead).
- The system **provisions the client's portal account** (``client_provisioning`` service) and **sends the credential email** (Brevo): thank-you message, portal URL, username, temporary password, login instructions.
- Result: ``Client`` record + client portal user; lead status ``converted`` with ``converted_client_id`` set.

Stage 4 — Proposal Workflow (Employee → PM Review → Client)

```
stateDiagram-v2
    [*] --> draft
    draft --> submitted_for_review : employee submits
    submitted_for_review --> pm_approved : PM approves
    submitted_for_review --> pm_rejected : PM rejects (review notes)
    pm_rejected --> draft : rework
    draft --> sent : sent directly by authorised staff
    pm_approved --> sent : sent to client (sent_at)
    sent --> viewed : client opens it
    sent --> accepted : client accepts
    viewed --> accepted : client accepts
    sent --> rejected : client rejects (rejection_reason)
    viewed --> rejected : client rejects (rejection_reason)
    rejected --> superseded : revision issued – new version
    accepted --> [*]
```

```
sequenceDiagram
    autonumber
    actor E as Employee / Staff
    actor PM as Project Manager
    actor C as Client
    participant S as System
    E->>S: Create draft proposal (version N)
```

```

S-->>E: draft · lead = proposal_created
E->>S: Submit for PM review
S-->>E: submitted_for_review
PM->>S: Approve or reject
alt Approved
  S-->>E: pm_approved – sending enabled
  E->>S: Send proposal to client
  S-->>C: Email + portal notification
  S-->>E: sent (sent_at) · lead = proposal_sent
else Rejected
  S-->>E: pm_rejected (review notes)
end
C->>S: Open proposal
S-->>C: viewed
alt Client accepts
  C->>S: Accept
  S-->>C: accepted · lead = proposal_approved
  S->>S: Auto-create project (planning, idempotent)
else Client rejects
  C->>S: Reject with reason
  S-->>C: rejected (rejection_reason)
  S->>S: Revision needed or lead disqualified
end

```

- Full proposal history is kept: version, created/sent dates, status, client comments, acceptance/rejection, rejection reason, approved version.

- Both staff-side and client-side accept paths run the same auto-provisioning (`project\_provisioning`) — idempotent, so a repeated accept never creates a second project.

Stage 5 — Contracts

```

stateDiagram-v2
  [*] --> pending : drafted from an accepted proposal
  pending --> signed : both parties signed
  pending --> void : cancelled
  signed --> [*]

```

- A contract can only be drafted from an **accepted** proposal (idempotent per proposal).

- Both parties sign (`signed\_by\_client\_at` + `signed\_by\_company\_at`); when both are present the contract becomes `signed`.

- On full signature the system can **provision the client account** (if not already provisioned) and **auto-create the project**, logging the conversion in the lead activity trail.

Stage 6 — Project Execution

```

stateDiagram-v2
  [*] --> planning
  planning --> in_progress : PM assigns employees – work begins
  in_progress --> on_hold : pause
  on_hold --> in_progress : resume
  in_progress --> completed : final review + client approval
  planning --> cancelled
  in_progress --> cancelled
  completed --> [*]

```

Task statuses:

```

stateDiagram-v2
  [*] --> todo
  todo --> in_progress
  in_progress --> in_review
  in_review --> done
  in_review --> in_progress : changes requested
  in_progress --> blocked
  blocked --> in_progress : unblocked
  done --> [*]

```

- **PM assigns employees** to the project; the PM coordinates requirements, assignments, progress, deliverables, communication and timelines.
- Tasks carry priorities: low / medium / high / urgent. Milestones and deliverables are tracked per project.
- **Daily updates:** employees record work completed and progress; the PM sees everything, and the client sees the approved, client-facing view through their portal (same data, permission-filtered).
- Timesheets: `draft → submitted → approved / rejected`.

Stage 7 — Documents & Meetings

- **Documents:** dedicated page in every portal; versioned with name, type, uploader, date, status and access permissions (architecture docs, PRD/TRD, design docs, deliverables, HR docs, etc.). Upload/download gated by permissions.
- **Meetings:** client ↔ PM/employee; statuses `scheduled → completed / cancelled`. Scheduling sends the client both an **in-app** notification and an email; meetings carry title, date/time, participants, link/location, agenda.

Stage 8 — Completion → Payment → Invoice

```
stateDiagram-v2
    direction LR
    [*] --> draft
    draft --> sent : issued to the client
    sent --> paid : payment completed
    sent --> overdue : past due date (sweep-overdue)
    overdue --> paid : payment completed
    sent --> cancelled
    paid --> [*]
```

```
stateDiagram-v2
    direction LR
    [*] --> pending
    pending --> completed
    pending --> failed
    completed --> refunded
    completed --> [*]
```

1. Final review → project `completed`; final deliverables shared with the client.
2. Client reviews and approves the completed work.
3. **Invoices:** total = amount + tax. `POST /invoices/sweep-overdue` flips `sent` invoices past their due date to `overdue` (manual/cron trigger — the app has no background job runner).
4. **Payments:** methods bank_transfer / card / upi / paypal / cheque / other. Recorded per invoice with an idempotency guarantee on `(invoice\_id, transaction\_ref)`.
5. The invoice (and full billing history) is available for viewing/download in the **Client Portal → Invoices / Payments**.

Stage 9 — Support / Tickets

```
flowchart LR
    A["Ticket created in the Client or Partner Portal"] --> B["open"]
    B --> C["in_progress – support assigned"]
    C --> D["resolved"]
    D --> E["closed"]
    B --> F["critical 4h · high 8h<br/>medium 24h · low 40h<br/>business hours only<br/>Mon-Fri"]
```

Priority-based SLA (business hours only — weekends and after-hours are skipped; `app/utls/sla.py`):

| Priority | Resolution target |

|---|---|

| Critical | 4 business hours |

| High | 8 business hours (1 business day) |

| Medium | 24 business hours (3 business days) |

| Low | 40 business hours (5 business days) |

3. Employee Portal Flow

flowchart TD

```
L["Employee login"] --> D["Dashboard"]
D --> ATT["Attendance – check-in / check-out<br/>(HR can read all, employee sees own)"]
D --> LV["Leave – apply"]
LV --> LVR["HR review"]
LVR -->|"approved"| N1["Notification to employee"]
LVR -->|"rejected"| N2["Notification to employee"]
LV --> LVC["cancelled (by employee)"]
D --> PR["Assigned projects & tasks"]
PR --> UP["Daily project updates"]
PR --> TS["Timesheets – draft → submitted → approved / rejected"]
PR --> PT["Performance (reviews, goals, ratings)<br/>& Training (assigned, completion)"]
D --> PS["Payslips – own only<br/>generated → paid"]
D --> DOC["Documents (employee / HR / policies)"]
D --> NP["Notifications & Profile"]
```

4. Admin Panel & CMS

- Admin manages: dashboard, users, clients, employees, leads, projects, proposals, tasks, meetings, documents, payments, invoices, support tickets, HR, notifications, reports, audit logs, system settings (and backup triggers).
- **CMS** for the public website: services, industries, blogs (draft → published → archived), FAQs, case studies, gallery, portfolio, careers, page content, announcements.
- Published CMS content instantly appears on the corresponding public page (CMS → Publish Blog → Public Blog Page).
- **SEO** per page: title, meta description, slug, canonical, Open Graph, robots, schema; sitemap served at `/sitemap.xml`.
- **Services** are single-sourced: created in CMS → selectable in the contact form → carried through lead → proposal → project.
- **Careers**: job openings (open/closed) with an application pipeline applied → shortlisted → interview → offered → rejected / hired.

5. Partner Portal (additional surface)

Partner accounts (partner role) have their own portal with the same pattern as clients: own projects/tickets/documents via `/partner/me/*` endpoints, including support tickets under the same SLA rules.

6. Platform Overview

flowchart TB

```
subgraph PUB["Public Website"]
    W1["Services · Industries · Blogs<br/>Case Studies · Careers · SEO"]
    W2["Contact Form → ContactSubmission"]
end
subgraph ADM["Admin Panel – staff"]
    A1["Leads · Clients · Employees<br/>Projects · Proposals · Contracts"]
    A2["Finance · Tickets · HR<br/>CMS · SEO · Reports · Audit · Settings"]
end
subgraph CLIP["Client Portal"]
    C1["Dashboard · Projects · Proposals"]
    C2["Documents · Meetings · Tickets"]
    C3["Invoices · Payments · Notifications"]
end
```

```

end
subgraph EMPP["Employee Portal"]
  E1["Attendance · Leave · Timesheets"]
  E2["Projects · Tasks · Performance"]
  E3["Training · Payslips · Documents"]
end
subgraph PARTP["Partner Portal"]
  P1["Partner projects · Tickets"]
end

W2 --> A1
A1 <--> CLIP
A1 <--> EMPP
A1 <--> PARTP

```

7. Status Reference (implemented enums)

| Entity | Statuses |

|---|---|

| Lead | new, contacted, requirement_gathering, proposal_created, proposal_sent, proposal_approved, converted, disqualified |

| Proposal | draft, submitted_for_review, pm_approved, pm_rejected, sent, viewed, accepted, rejected, superseded |

| Contract | pending, signed, void |

| Project | planning, in_progress, on_hold, completed, cancelled |

| Task | todo, in_progress, in_review, done, blocked |

| Timesheet | draft, submitted, approved, rejected |

| Leave | pending, approved, rejected, cancelled |

| Invoice | draft, sent, paid, overdue, cancelled |

| Payment | pending, completed, failed, refunded |

| Ticket | open, in_progress, resolved, closed |

| Contact submission | new, in_progress, resolved, spam |

| Blog | draft, published, archived |

| Job application | applied, shortlisted, interview, offered, rejected, hired |

8. Where It Lives in Code

- **Routers** (`backend/app/routers/`): ``contact``, ``leads``, ``proposals``, ``contracts``, ``clients``, ``partner_account``, ``projects``, ``task``, ``finance``, ``ticket``, ``employees``, ``meetings``, ``career``, ``blog``, ...
- **Services** (`backend/app/services/`): ``client_provisioning`` (account + credentials), ``project_provisioning`` (project from accepted proposal), ``lead_pipeline`` (lead activity/status transitions), ``notification_service``, ``email_service`` (Brevo).
- **SLA helpers**: `backend/app/utls/sla.py`` (business-hours clock).
- **Spec source**: `docs/requirments/Complete_Website_Portal_Workflow.md``; per-domain detail in `docs/modules/*`` and `docs/flows/*``.